

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



Đồ án môn học Thiết kế luận lý (CO3091)
Đề tài: Hiện thực bộ mã hóa AES với Verilog

GVHD: PGS.TS Phạm Quốc Cường
Lớp TN01

Nguyễn Văn Hùng – 2311301
Lại Nguyễn Hoàng Hưng – 2311327

TP. Hồ Chí Minh, Tháng 12 - 2025



Mục lục

1	Tổng quan	1
1.1	Lịch sử hình thành	1
1.2	Đặc điểm	1
1.3	Mục tiêu thiết kế	1
2	Thiết kế	1
2.1	Quy trình	1
2.2	Mô tả	2
3	Giải thuật	2
3.1	addRoundKey()	2
3.2	subBytes() – invSubBytes()	3
3.3	shiftRows() – invShiftRows()	3
3.4	mixColumns() – invMixColumns()	5
3.5	keyExpansion()	6
3.6	Mã hóa và giải mã	6
4	Unrolled AES	8
4.1	Nguyên lý thiết kế	8
4.2	Sơ đồ RTL	9
4.3	Mô phỏng	10
5	Iterative AES	11
5.1	Nguyên lý thiết kế	11
5.2	Sơ đồ RTL	12
5.3	Mô phỏng	15
6	Pipelined AES	15
6.1	Nguyên lý thiết kế	15
6.2	Sơ đồ RTL	16
6.3	Mô phỏng	17
7	Tổng hợp và so sánh kết quả	18
7.1	Tổng quan về phần tổng hợp	18
7.2	Tài nguyên phần cứng	19
7.3	Hiệu suất thời gian	20
7.4	Công suất tiêu thụ (Power Consumption)	21
8	Kết luận và Hướng phát triển	22



Danh mục hình ảnh

2.1	Lưu đồ thể hiện quy trình thiết kế	2
3.1	Minh họa module <code>subBytes()</code>	3
3.2	Minh họa phép biến đổi <code>shiftRows()</code>	4
3.3	Minh họa phép biến đổi <code>MixColumns()</code>	5
3.4	Lưu đồ thuật toán <code>keyExpansion()</code>	7
3.5	Mã hóa và giải mã AES	8
4.1	Sơ đồ toàn kiến trúc Unrolled	9
4.2	Khối <code>keyExpansion</code>	9
4.3	Các khối <code>encryptRound</code>	9
4.4	Các khối của vòng lặp cuối cùng	9
4.5	Kết quả chạy testbench mã hóa	10
4.6	Kết quả chạy testbench giải mã	11
5.1	AES_Iterative FSM	12
5.2	Sơ đồ của toàn kiến trúc Iterative	13
5.3	Sơ đồ của phần xử lý <code>round_cnt</code>	13
5.4	Sơ đồ phần xử lý <code>key</code> và các <i>round</i>	13
5.5	Sơ đồ phần xử lý cho <code>done</code> và kết quả	14
5.6	Kết quả mô phỏng với kiến trúc Iterative	15
6.1	Sơ đồ của toàn kiến trúc Pipeline	16
6.2	Sơ đồ <code>key</code> và các thanh ghi trong Pipeline	16
6.3	Sơ đồ của <code>round</code> cuối cùng trong Pipeline	16
6.4	Kết quả mô phỏng cho kiến trúc Pipeline với key 128-bit	17
6.5	Kết quả mô phỏng cho kiến trúc Pipeline với key 192-bit	17
6.6	Kết quả mô phỏng cho kiến trúc Pipeline với key 256-bit	18
7.1	Biểu đồ so sánh tài nguyên tiêu thụ giữa 3 kiến trúc	19
7.2	Công suất tiêu thụ của các kiến trúc	21



Danh mục bảng

1	Tổ hợp Key–Block–Round	3
2	Sbox() : Các giá trị thay thế cho byte xy (Dạng thập lục phân)	4
3	Các hằng số vòng (Round constants)	6
4	Bảng tổng hợp hiệu năng thời gian	20
5	Bảng so sánh đặc tính kỹ thuật giữa các kiến trúc AES-128	22

1 Tổng quan

Trong kỷ nguyên số hóa, việc bảo mật dữ liệu trên các kênh truyền thông và thiết bị lưu trữ đã trở thành một yêu cầu cấp thiết. Để đáp ứng nhu cầu này, Chuẩn mã hóa tiên tiến (Advanced Encryption Standard - AES) đã được thiết lập như một trong những thuật toán mật mã quan trọng và phổ biến nhất trên thế giới.

1.1 Lịch sử hình thành

Thuật toán AES, tiền thân là thuật toán Rijndael, được phát triển bởi hai nhà mật mã học người Bỉ là Joan Daemen và Vincent Rijmen. Sau một quá trình tuyển chọn khắt khe kéo dài 5 năm nhằm thay thế chuẩn mã hóa cũ DES (Data Encryption Standard), Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ (NIST) đã chính thức công bố AES là tiêu chuẩn mã hóa dữ liệu vào ngày 26/11/2001 (theo ấn bản FIPS PUB 197). Tại Việt Nam, thuật toán này cũng đã được chuẩn hóa thông qua tiêu chuẩn quốc gia TCVN 7816:2007 vào năm 2007.

1.2 Đặc điểm

Về mặt kỹ thuật, AES là một thuật toán mã hóa khối đối xứng (Symmetric Block Cipher). Thuật toán này hoạt động trên các khối dữ liệu có kích thước cố định là 128 bits. Tùy thuộc vào yêu cầu về mức độ an toàn và hiệu năng, AES hỗ trợ ba biến thể với độ dài khóa khác nhau:

- AES-128: Sử dụng khóa 128 bits.
- AES-192: Sử dụng khóa 192 bits.
- AES-256: Sử dụng khóa 256 bits.

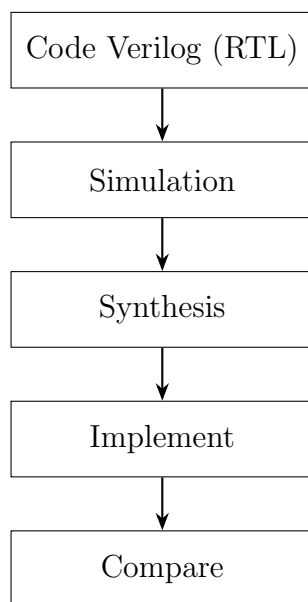
1.3 Mục tiêu thiết kế

- Hiện thực thuật toán AES-128 trên phần cứng theo ba kiến trúc: Kiến trúc lặp (Iterative), Kiến trúc trải phẳng (Unrolled), Kiến trúc đường ống (Pipeline)
- Đảm bảo tính đúng đắn chức năng của thuật toán
- Đánh giá và so sánh hiệu năng giữa các kiến trúc

2 Thiết kế

2.1 Quy trình

Hình 2.1 thể hiện quy trình hiện thực. Đồ án này sẽ hiện thực bằng phần mềm Vivado.



Hình 2.1: Lưu đồ thể hiện quy trình thiết kế

2.2 Mô tả

- **Code Verilog (RTL):** Hiện thực thuật toán AES ở mức RTL bằng ngôn ngữ Verilog, mô tả cấu trúc phần cứng và luồng dữ liệu của các kiến trúc khác nhau.
- **Simulation:** Thực hiện mô phỏng chức năng bằng testbench để kiểm tra tính đúng đắn của thiết kế, đảm bảo kết quả mã hóa và giải mã phù hợp với giá trị mong đợi.
- **Synthesis:** Tổng hợp thiết kế RTL sang các phần tử logic của FPGA, từ đó thu thập các thông số về tài nguyên phần cứng và timing sơ bộ.
- **Implementation:** Thực hiện đặt và định tuyến (place and route) để đánh giá chính xác các đặc tính timing và khả năng hoạt động của thiết kế trên FPGA mục tiêu.
- **Compare:** So sánh các kiến trúc dựa trên các tiêu chí như diện tích, tần số hoạt động, độ trễ và thông lượng, nhằm đánh giá ưu và nhược điểm của từng phương pháp thực hiện.

3 Giải thuật

3.1 addRoundKey()

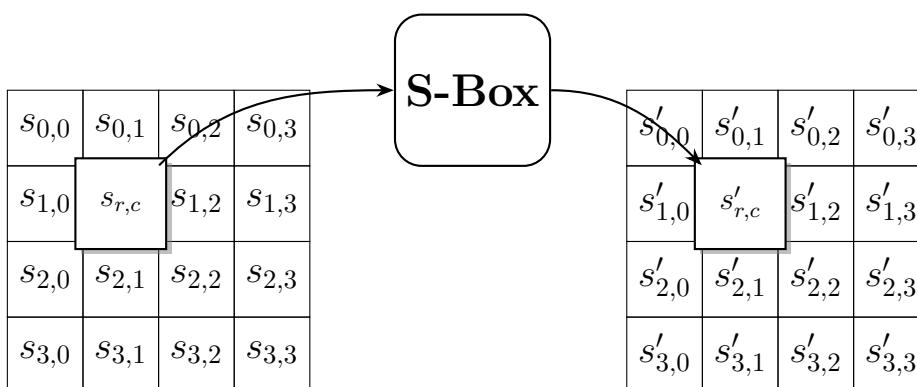
Phép biến đổi trạng thái, lấy kết quả phép xor giữa `round key` và `state`.

Bảng 1: Tổ hợp Key–Block–Round

	Độ dài Key Nk (bit)		Block size Nk (bit)		Số vòng Round Nr
AES-128	4	128	4	128	10
AES-192	6	192	4	128	12
AES-256	8	256	4	128	14

3.2 subBytes() – invSubBytes()

Hình 3.1 minh họa cách hàm subBytes() biến đổi trạng thái.



Hình 3.1: Minh họa module subBytes()

S-box được trình bày trong Bảng 2. Với SubBytes() của mã hóa, khi byte đầu vào có giá trị $s_{r,c} = \{53\}$, lấy kết quả tại hàng '5' và cột '3' ta có kết quả $s'_{r,c} = \{ed\}$.

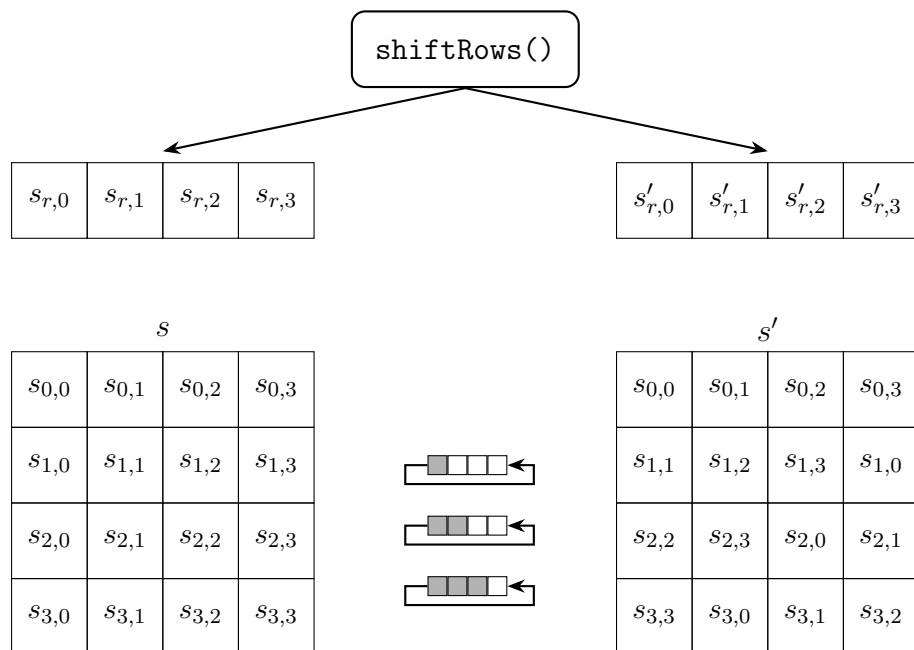
invSubBytes() của quá trình giải mã sẽ tra ngược lại kết tương ứng với bảng trên. Ví dụ với byte đầu là $\{f7\}$, kết quả output sẽ là $\{82\}$.

3.3 shiftRows() – invShiftRows()

shiftRows() được minh họa trong Hình 3.2. Trong cách biểu diễn trạng thái này, phép biến đổi dịch chuyển mỗi byte sang trái r vị trí trong hàng tương ứng, đồng thời xoay vòng r byte ở phía trái sang cuối hàng bên phải. Hàng thứ nhất, với $r = 0$, không thay đổi.

Bảng 2: Sbox(): Các giá trị thay thế cho byte xy (Dạng thập lục phân)

		y															
		0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
x	0	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
	1	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
	2	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
	3	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
	4	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
	5	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
	6	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
	7	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
	8	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
	9	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
	a	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
	b	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
	c	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
	d	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
	e	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
	f	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16



Hình 3.2: Minh họa phép biến đổi shiftRows()

Với phép biến đổi invShiftRows() của giải mã, ta thực hiện các bước tương tự. Nhưng

quá trình dịch phải sẽ thay bằng dịch trái để lấy lại kết quả ban đầu.

3.4 mixColumns() – invMixColumns()

mixColumns() là một phép biến đổi của *state*, trong đó mỗi cột trong bốn cột của *state* được nhân với cùng một ma trận cố định. Các phần tử của ma trận này được lấy từ word sau:

$$[a_0, a_1, a_2, a_3] = [\{02\}, \{01\}, \{01\}, \{03\}]. \quad (3.1)$$

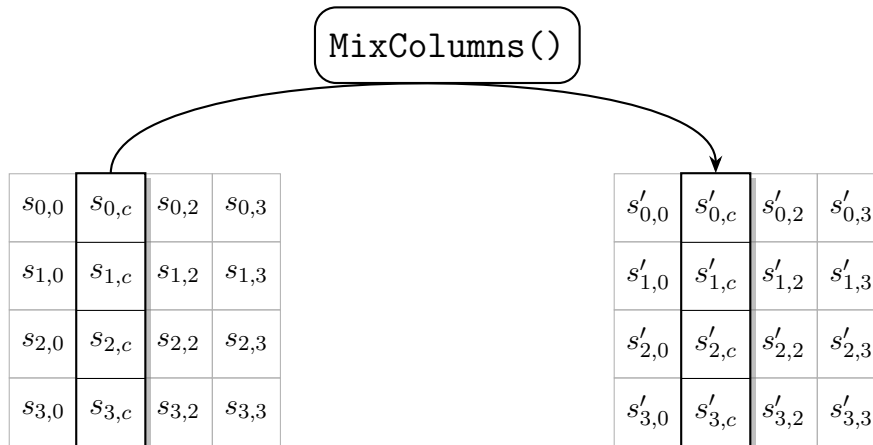
Do đó, phép biến đổi mixColumns() được biểu diễn như sau:

$$\begin{bmatrix} s'_{0,c} \\ s'_{1,c} \\ s'_{2,c} \\ s'_{3,c} \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_{0,c} \\ s_{1,c} \\ s_{2,c} \\ s_{3,c} \end{bmatrix}, \text{ với } 0 \leq c < 4. \quad (3.2)$$

Từ đó, các byte đầu ra riêng lẻ được xác định như sau:

$$\begin{aligned} s'_{0,c} &= (\{02\} \cdot s_{0,c}) \oplus (\{03\} \cdot s_{1,c}) \oplus s_{2,c} \oplus s_{3,c}, \\ s'_{1,c} &= s_{0,c} \oplus (\{02\} \cdot s_{1,c}) \oplus (\{03\} \cdot s_{2,c}) \oplus s_{3,c}, \\ s'_{2,c} &= s_{0,c} \oplus s_{1,c} \oplus (\{02\} \cdot s_{2,c}) \oplus (\{03\} \cdot s_{3,c}), \\ s'_{3,c} &= (\{03\} \cdot s_{0,c}) \oplus s_{1,c} \oplus s_{2,c} \oplus (\{02\} \cdot s_{3,c}). \end{aligned} \quad (3.3)$$

Hình 3.3 minh họa phép biến đổi mixColumns().



Hình 3.3: Minh họa phép biến đổi MixColumns()

Module invMixColumns() của giải mã, sẽ thực hiện chuyển đổi tương tự với đầu vào và ma trận khác được lấy từ word sau:

$$[a_0, a_1, a_2, a_3] = [\{0e\}, \{09\}, \{0d\}, \{0b\}]. \quad (3.4)$$

3.5 keyExpansion()

Giải thuật `keyExpansion()` lấy Khóa (`key`) làm đầu vào và tạo ra một mảng tuyến tính các từ (words), ký hiệu là $w[i]$, với tổng số từ là $4(Nr + 1)$. Giải thuật này sử dụng các hằng số vòng (round constants), ký hiệu là $Rcon[j]$, giá trị $Rcon[j]$ được cho dưới dạng hexa trong Bảng 3:

Bảng 3: Các hằng số vòng (Round constants)

j	$Rcon[j]$	j	$Rcon[j]$
1	[01, 00, 00, 00]	6	[20, 00, 00, 00]
2	[02, 00, 00, 00]	7	[40, 00, 00, 00]
3	[04, 00, 00, 00]	8	[80, 00, 00, 00]
4	[08, 00, 00, 00]	9	[1b, 00, 00, 00]
5	[10, 00, 00, 00]	10	[36, 00, 00, 00]

Trong quá trình mở rộng khóa (`keyExpansion()`), có hai phép biến đổi được áp dụng lên các từ (word), bao gồm `rotWord()` và `subWord()`. Cho một từ đầu vào được biểu diễn dưới dạng một dãy bốn byte $[a_0, a_1, a_2, a_3]$, phép biến đổi `rotWord()` thực hiện việc xoay vòng các byte trong từ theo quy tắc:

$$\text{rotWord}([a_0, a_1, a_2, a_3]) = [a_1, a_2, a_3, a_0]. \quad (3.5)$$

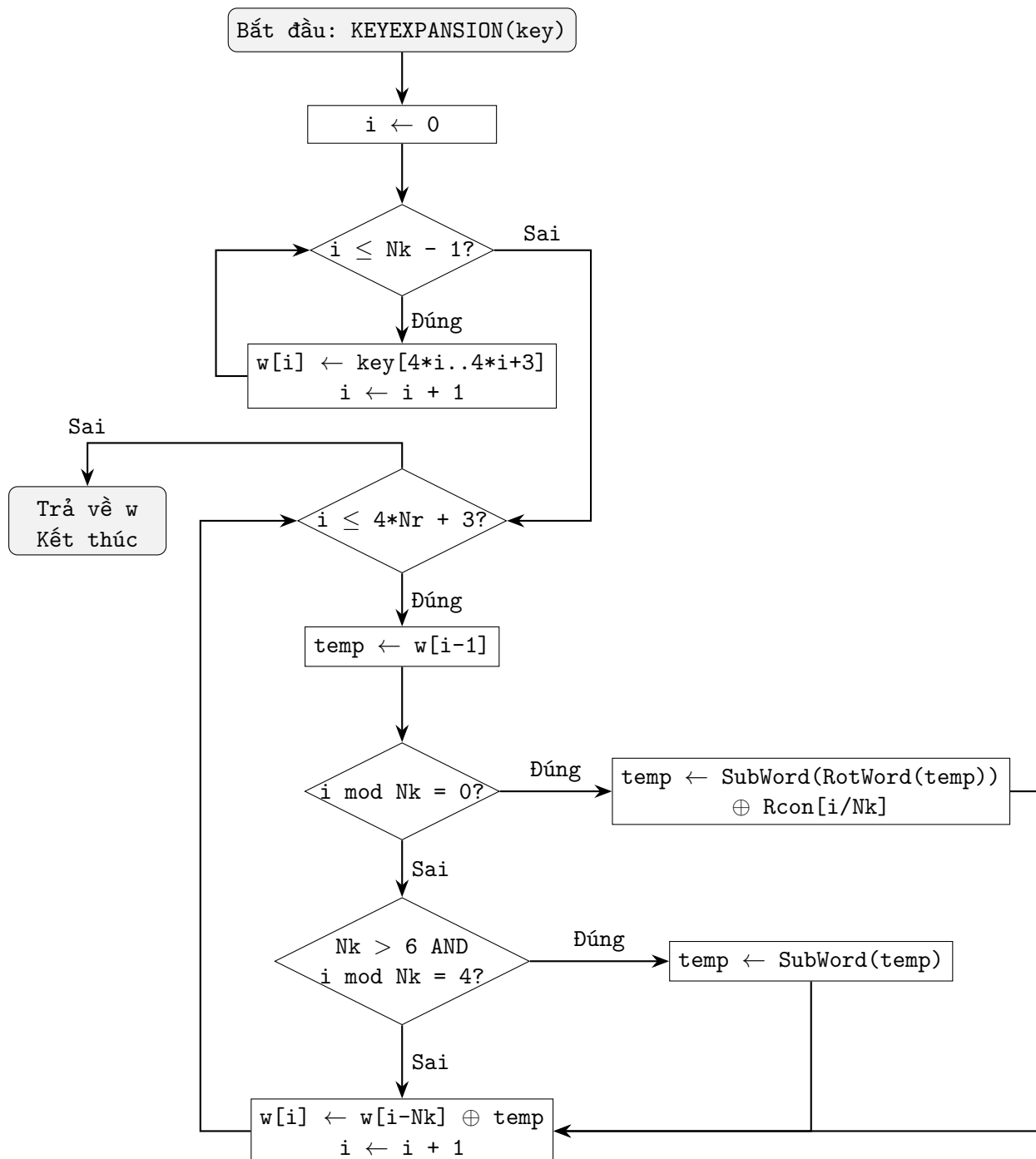
Phép biến đổi `subWord()` thực hiện việc thay thế từng byte của từ đầu vào bằng giá trị tương ứng trong bảng thế S-box, cụ thể:

$$\text{subWord}([a_0, a_1, a_2, a_3]) = [\text{SBox}(a_0), \text{SBox}(a_1), \text{SBox}(a_2), \text{SBox}(a_3)]. \quad (3.6)$$

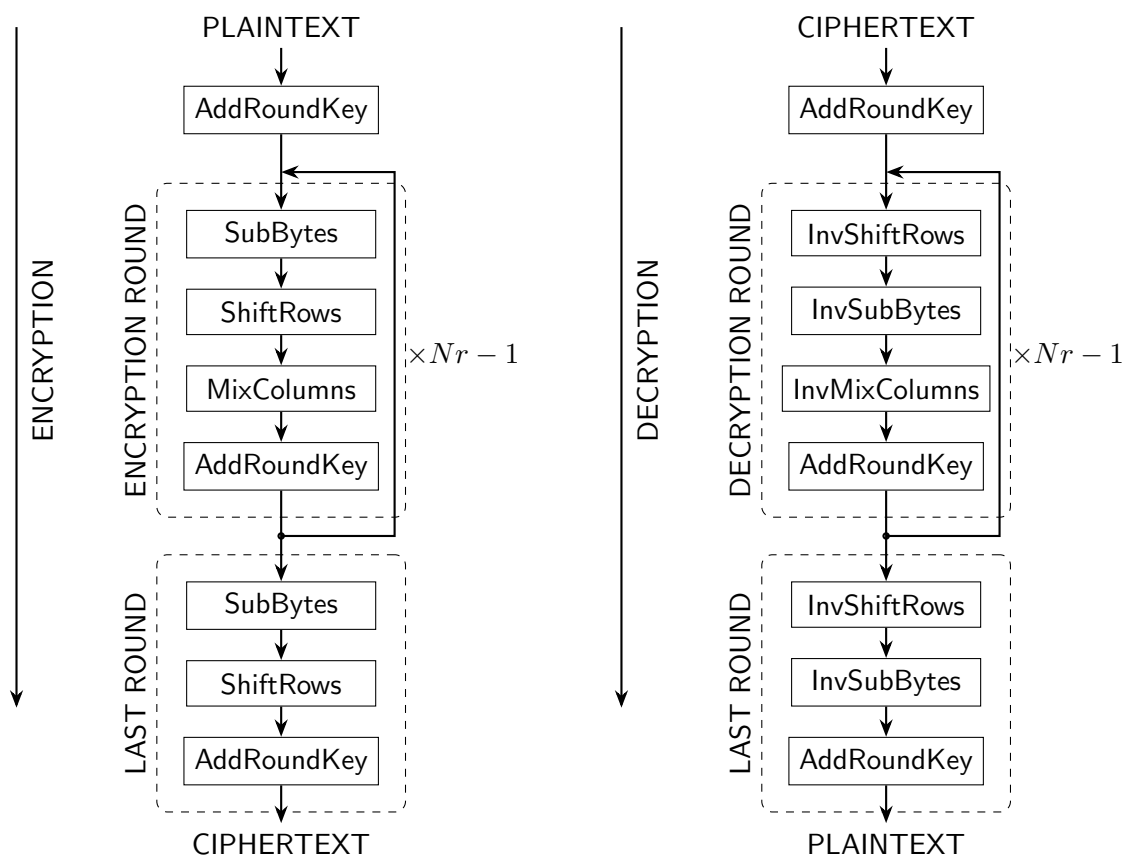
Thuật toán `keyExpansion()` được thể hiện trong Hình 3.4.

3.6 Mã hóa và giải mã

Luồng xử lý của mã hóa và giải mã được mô tả trong Hình 3.5.



Hình 3.4: Lưu đồ thuật toán `keyExpansion()`



Hình 3.5: Mã hóa và giải mã AES

4 Unrolled AES

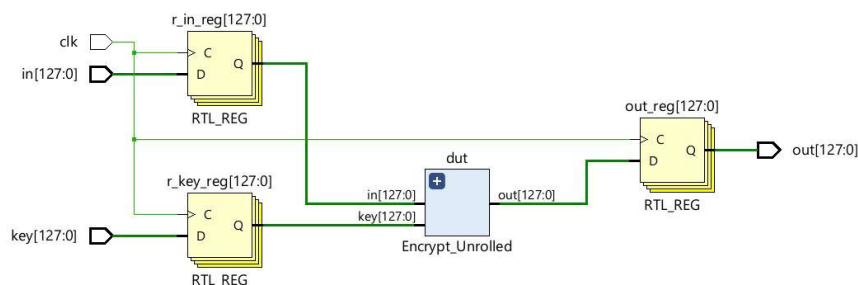
4.1 Nguyên lý thiết kế

Kiến trúc AES Unrolled được xây dựng bằng cách trải phẳng toàn bộ các vòng mã hóa của thuật toán AES thành các khối phần cứng độc lập. Mỗi vòng mã hóa được hiện thực bằng một khối logic riêng và các khối này được kết nối nối tiếp trực tiếp, tạo thành một chuỗi xử lý cố định từ đầu vào đến đầu ra.

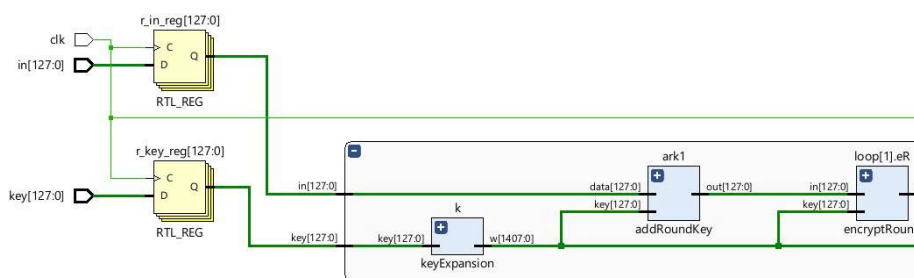
Trong kiến trúc này, không sử dụng cơ chế chèn thanh ghi trung gian giữa các vòng. Dữ liệu đầu vào lan truyền liên tục qua toàn bộ các khối vòng mã hóa và kết quả được tạo ra sau khi tín hiệu đi qua tất cả các vòng trong một lần xử lý.

Mục tiêu của kiến trúc Unrolled là giảm thiểu độ trễ mã hóa, đổi lại là độ dài đường đi tín hiệu lớn và mức sử dụng tài nguyên phần cứng cao.

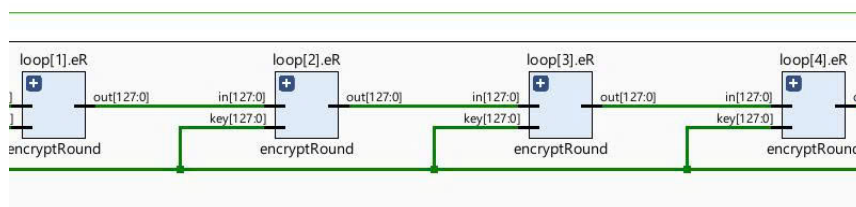
4.2 Sơ đồ RTL



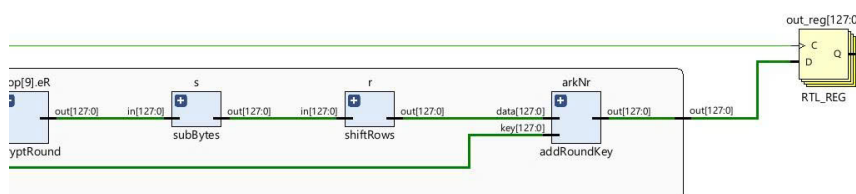
Hình 4.1: Sơ đồ toàn kiến trúc Unrolled



Hình 4.2: Khối keyExpansion



Hình 4.3: Các khối encryptRound



Hình 4.4: Các khối của vòng lặp cuối cùng

Hình 4.2 thể hiện khối **keyExpansion** chịu trách nhiệm sinh toàn bộ **round key** ngay từ đầu chu trình.

- Sinh khóa một lần, sử dụng song song: Khối `keyExpansion` tạo ra toàn bộ tập khóa con (round keys) và phân phối trực tiếp đến các khối `encryptRound`.
- Không phụ thuộc dữ liệu mã hóa: Việc sinh khóa hoàn toàn độc lập với dữ liệu `plaintext` → phù hợp với kiến trúc Unrolled.

Hình 4.3 thể hiện chuỗi các module `encryptRound` được nhân bản nhiều lần và mắc nối tiếp (9 khối cho AES-128).

- Mỗi `encryptRound` tương ứng chính xác một vòng AES.
- Kết nối thuần tổ hợp: Đầu ra của round trước nối trực tiếp vào đầu vào round sau, không qua thanh ghi trung gian.
- Round key được cấp riêng cho từng `encryptRound` → loại bỏ hoàn toàn sự phụ thuộc thời gian giữa các vòng.

Điều này khẳng định rõ ràng rằng kiến trúc được tối ưu độ trễ, chấp nhận tăng độ dài critical path.

Các khối thực thi vòng xử lý cuối cùng cũng được tách ra riêng biệt như Hình 4.4.

4.3 Mô phỏng

```
=====
                        AES UNROLLED ENCRYPTION TEST
=====

>> AES-128 Encryption (NIST)
-----
Plaintext      : 3243f6a8885a308d313198a2e0370734
Key            : 2b7e151628aed2a6abf7158809cf4f3c
Ciphertext (DUT) : 3925841d02dc09fbdcl18597196a0b32
Expected Ciphertext: 3925841d02dc09fbdcl18597196a0b32
Result         : PASS

>> AES-192 Encryption
-----
Plaintext      : 3243f6a8885a308d313198a2e0370734
Key            : 000102030405060708090a0b0c0d0e0f1011121314151617
Ciphertext (DUT) : bc3aaab5d97baa7b325d7b8f69cd7ca8
Expected Ciphertext: bc3aaab5d97baa7b325d7b8f69cd7ca8
Result         : PASS

>> AES-256 Encryption
-----
Plaintext      : 3243f6a8885a308d313198a2e0370734
Key            : 000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
Ciphertext (DUT) : 9a198830ff9a4e39ec1501547d4a6b1b
Expected Ciphertext: 9a198830ff9a4e39ec1501547d4a6b1b
Result         : PASS

=====
                        ENCRYPTION TEST COMPLETED
=====
```

Hình 4.5: Kết quả chạy testbench mã hóa

Kết quả mô phỏng từ 2 Hình 4.5 và 6.6 cho thấy khối mã hóa và giải mã AES hoạt động đúng chức năng, khi dữ liệu đầu ra sau quá trình giải mã trùng khớp hoàn toàn với dữ liệu đầu vào ban đầu.

```
=====
                        AES UNROLLED DECRYPTION TEST
=====

>> AES-128 Decryption (NIST)
-----
Ciphertext (Input) : 3925841d02dc09fdbc118597196a0b32
Key                : 2b7e151628aed2a6abf7158809cf4f3c
Plaintext (DUT)    : 3243f6a8885a308d313198a2e0370734
Expected Plaintext : 3243f6a8885a308d313198a2e0370734
Result             : PASS

>> AES-192 Decryption
-----
Ciphertext (Input) : bc3aaab5d97baa7b325d7b8f69cd7ca8
Key                : 000102030405060708090a0b0c0d0e0f1011121314151617
Plaintext (DUT)    : 3243f6a8885a308d313198a2e0370734
Expected Plaintext : 3243f6a8885a308d313198a2e0370734
Result             : PASS

>> AES-256 Decryption
-----
Ciphertext (Input) : 9a198830ff9a4e39ec1501547d4a6b1b
Key                : 000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
Plaintext (DUT)    : 3243f6a8885a308d313198a2e0370734
Expected Plaintext : 3243f6a8885a308d313198a2e0370734
Result             : PASS

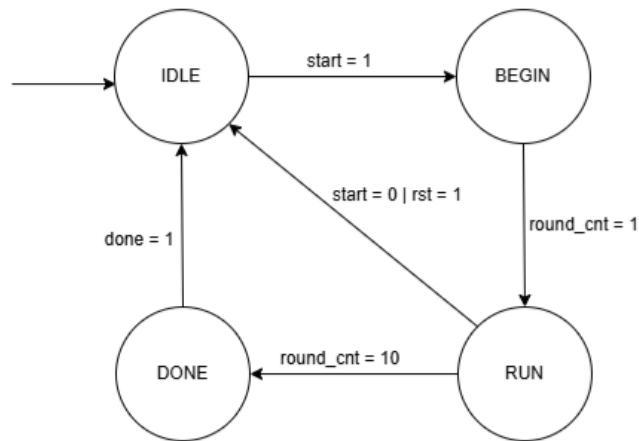
=====
                        DECRYPTION TEST COMPLETED
=====
```

Hình 4.6: Kết quả chạy testbench giải mã

5 Iterative AES

5.1 Nguyên lý thiết kế

Kiến trúc Iterative được thiết kế nhằm tiết kiệm tài nguyên phần cứng. Thay vì sử dụng N_r khối logic cho N_r vòng lặp mã hóa, thiết kế này chỉ sử dụng duy nhất một khối tổ hợp `encryptRound` và tái sử dụng nó trong nhiều chu kỳ xung nhịp. Điều này giúp tối ưu hóa diện tích nhưng đánh đổi bằng việc tăng độ trễ (Latency), do cần nhiều chu kỳ clock để hoàn thành một khối dữ liệu.



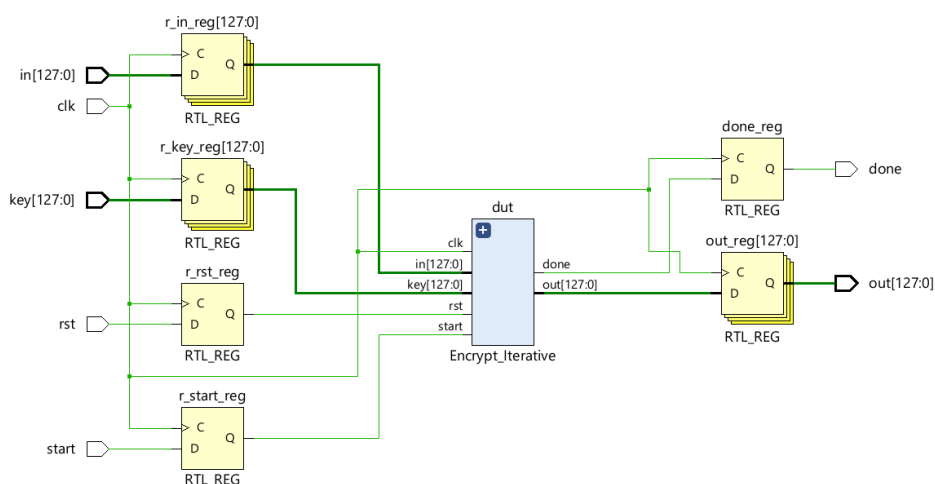
Hình 5.1: AES_Iterative FSM

Máy trạng thái mô tả nguyên lý hoạt động của kiến trúc trong Hình 6.6 cho thấy: Việc thực thi của kiến trúc này sẽ đồng bộ theo xung clock (*clk*) với các trạng thái bao gồm:

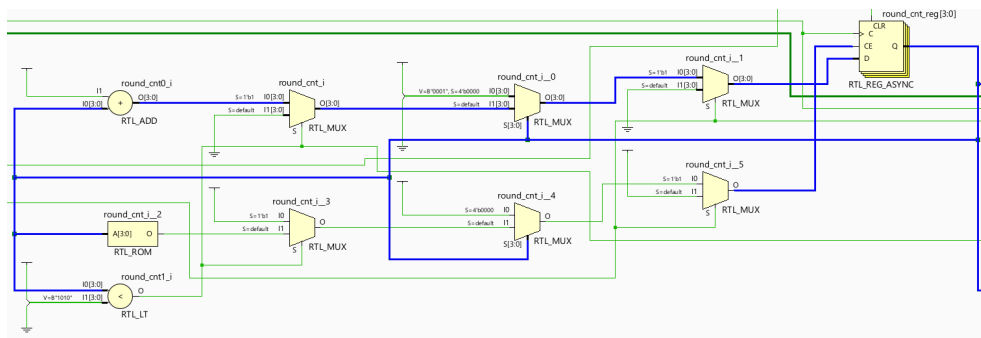
- **IDLE:** Đây là trạng thái nghỉ cũng như là trạng thái khởi tạo ban đầu của kiến trúc. Tại đây, *round_cnt*, *start* và các giá khác của hệ thống sẽ là 0. Khi có tín hiệu *start* = 1 thì lập tức chuyển sang BEGIN.
- **BEGIN:** Tại đây hệ thống thực hiện bước đầu tiên của quá trình mã hóa là *AddRoundKey*, sau đó nâng *round_cnt* lên 1, hệ thống sẽ chuyển sang RUN khi có tín hiệu cạnh lên từ *clk*.
- **RUN:** Ở trạng thái này, một vòng 4 bước của quá trình mã hóa sẽ diễn ra và giá trị *round_cnt* sẽ tăng lên 1 đơn vị, tất cả trong một xung clock. Quá trình đó sẽ lặp đi lặp lại cho đến khi *round_cnt* = 10 và chuyển sang DONE. Tuy nhiên trong trạng thái này, nếu có tín hiệu *rst* ở bất kỳ thời điểm cạnh lên nào của *clk* thì hệ thống sẽ chuyển về lại IDLE và đặt lại tất cả các giá trị về 0.
- **DONE:** Trạng thái này thực hiện vòng thứ 10 của quá trình mã hóa (khi *round_cnt* = 10), đặt giá trị *done* = 1. Đến khi cạnh lên tiếp theo thì hệ thống trở về IDLE cho đến khi tiếp tục có tín hiệu *start*.

5.2 Sơ đồ RTL

Sơ đồ RTL được tạo ra từ RTL Analysis của Vivado sẽ làm rõ hơn phần hiện thực cho kiến trúc này.

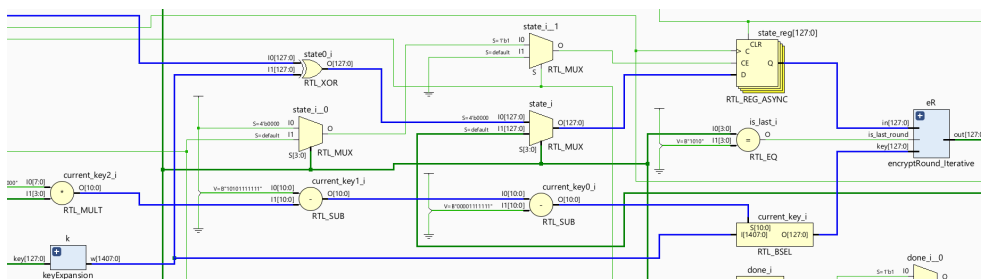


Hình 5.2: Sơ đồ của toàn kiến trúc Iterative



Hình 5.3: Sơ đồ của phần xử lý round_cnt

Dựa vào sơ đồ, ta có thể thấy có rất nhiều bộ chọn MUX được sinh ra do việc chuyển trạng thái của hệ thống đều dựa vào biến này, có bộ cộng, và cả khối so sánh được sinh ra để thực hiện cộng 1 mỗi vòng và so sánh xem đã đủ số vòng chưa.



Hình 5.4: Sơ đồ phần xử lý key và các round

Các key cần được xử lý để chọn ra cho từng vòng, vì key sau khi được expand sẽ đi qua các bộ nhân, trừ và bộ chọn Byte (RTL_BSEL) theo mã nguồn đã xây dựng:

```
state <= in ^ keySched[(128*(Nr+1)-1) -: 128];
```

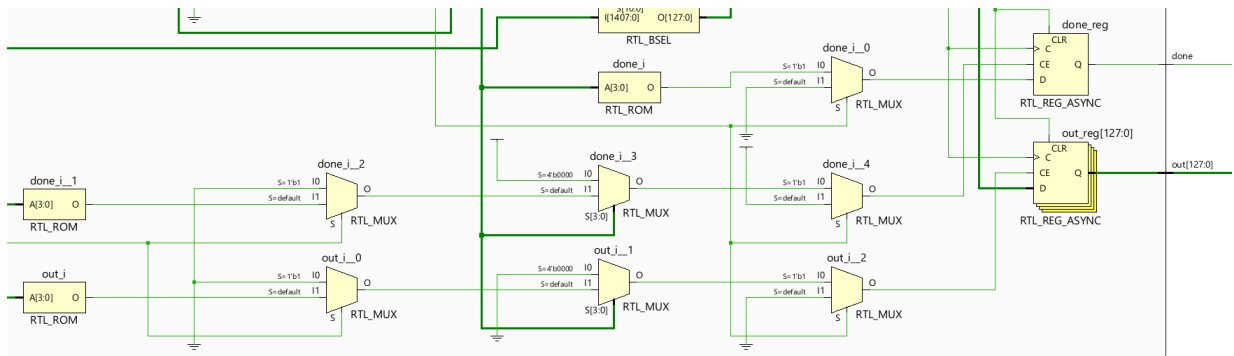
Bên cạnh đó là các bộ chọn cho state qua từng vòng, khi đã có *key* và *state* tương ứng cho mỗi vòng, khối *encryptRound_Iterative* sẽ thực hiện các bước mã hóa cho mỗi vòng.

Trong đó *encryptRound_Iterative* được hiện thực khác hơn một chút so với *encryptRound* thông thường, trong module này có thêm phần mã nguồn như sau:

```
assign data_before_addkey = (is_last_round == 1'b1)
? afterShiftRows : afterMixColumns;
addRoundKey b(data_before_addkey, key, out);
```

Nếu như là vòng cuối cùng *is_last_round* = 1 thì sẽ bỏ qua *MixColumns*, và lấy kết quả sau khi *shiftRows* để tiếp tục được *addRoundKey*.

Nếu sử dụng *encryptRound* thông thường, thì phần cứng được tạo ra sẽ có thêm 3 khối logic ở vòng lặp cuối được tạo ra riêng biệt gây ra lãng phí. Dùng cách chọn module được thực thi ở vòng cuối giúp cho tận dụng được 3 khối logic đã có sẵn trong *is_last_round*.



Hình 5.5: Sơ đồ phần xử lý cho *done* và kết quả

Sở dĩ *done* lại sinh ra nhiều bộ chọn như vậy là vì để *done* = 1 thì phải cần nhiều điều kiện như *start* phải là 1, *round_cnt* phải bằng *N_r*, còn khi *done* = 0 thì phải có tín hiệu *rst*, *round_cnt* = 0...

5.3 Mô phỏng

```
=====
                        AES ITERATIVE ENCRYPTION TEST
=====

>> AES-128 Encryption
-----
Plaintext      : 3243f6a8885a308d313198a2e0370734
Key            : 2b7e151628aed2a6abf7158809cf4f3c
Ciphertext (DUT) : 3925841d02dc09fdbc118597196a0b32
Expected Ciphertext: 3925841d02dc09fdbc118597196a0b32
Result         : PASS

>> AES-192 Encryption
-----
Plaintext      : 3243f6a8885a308d313198a2e0370734
Key            : 000102030405060708090a0b0c0d0e0f1011121314151617
Ciphertext (DUT) : bc3aaab5d97baa7b325d7b8f69cd7ca8
Expected Ciphertext: bc3aaab5d97baa7b325d7b8f69cd7ca8
Result         : PASS

>> AES-256 Encryption
-----
Plaintext      : 3243f6a8885a308d313198a2e0370734
Key            : 000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
Ciphertext (DUT) : 9a198830ff9a4e39ec1501547d4a6b1b
Expected Ciphertext: 9a198830ff9a4e39ec1501547d4a6b1b
Result         : PASS

=====
                        ENCRYPTION TEST COMPLETED
=====
```

Hình 5.6: Kết quả mô phỏng với kiến trúc Iterative

kết quả chạy thử cho thấy kiến trúc này được hiện thực đúng kết quả cho cả 3 độ dài của khóa.

6 Pipelined AES

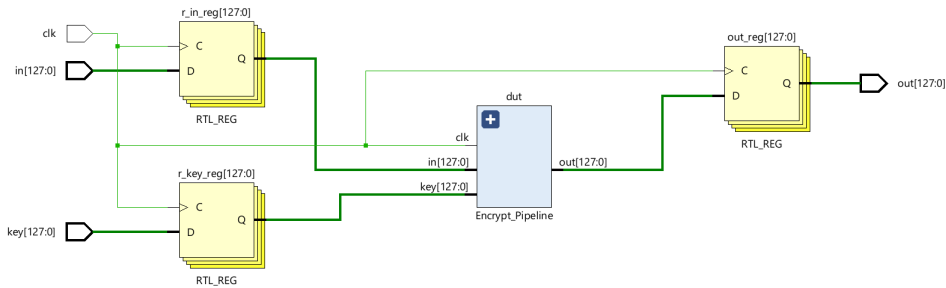
6.1 Nguyên lý thiết kế

Kiến trúc Pipeline được xây dựng để tối ưu hóa thời gian, xử lý từng bước của một quá trình, hết bước này tới bước kia liên tục và luân phiên cho lần. Khác với kiến trúc *Unrolled* (Trai phẳng toàn bộ mạch) hay *Iterative* (tái sử dụng một khối làm giảm băng thông), kiến trúc Pipeline thực hiện chia một quy trình ra làm nhiều bước với các cơ chế sau:

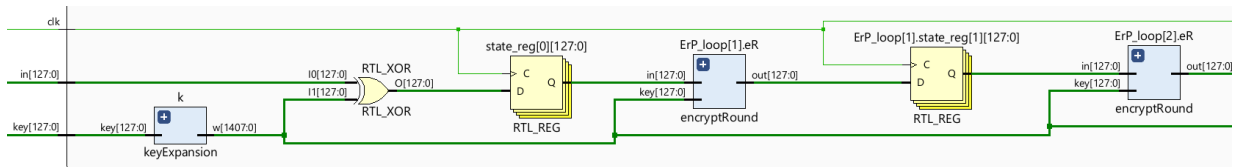
- **Chèn thanh ghi:** Hệ thống chèn các hàng thanh ghi vào giữa các khối logic của từng vòng mã hóa (Round) để lưu giữ trạng thái và truyền sang cho khối tiếp theo.
- **Xử lý gói đầu:** Tại một thời điểm, trong khi Vòng 10 đang xử lý gói dữ liệu A, thì Vòng 9 đang xử lý gói B, và Vòng 1 đang tiếp nhận gói K.
- **Mục tiêu:** Giảm chiều dài đường đi của tín hiệu trong một chu kỳ clock xuống chỉ còn bằng 1 vòng mã hóa.

6.2 Sơ đồ RTL

Sơ đồ RTL được tạo ra từ RTL Analysis của Vivado sẽ làm rõ hơn phần hiện thực cho kiến trúc này.

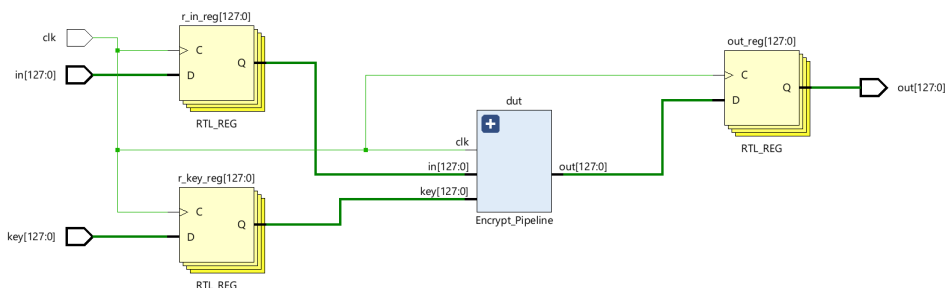


Hình 6.1: Sơ đồ của toàn kiến trúc Pipeline



Hình 6.2: Sơ đồ key và các thanh ghi trong Pipeline

Kiến trúc Pipeline thêm vào các thanh ghi để lưu giữ trạng thái cho mỗi bước xử lý giữa các round. Giá trị sẽ được truyền qua bước tiếp theo theo tín hiệu đồng bộ từ xung clk.



Hình 6.3: Sơ đồ của round cuối cùng trong Pipeline

Kiến trúc này cũng sinh ra 9 bộ **encryptRound** riêng biệt kết hợp với các thanh ghi để xử lý cho mỗi vòng. Vòng cuối cùng vẫn sẽ là 3 khối con tương ứng.

6.3 Mô phỏng

```
=====
                        AES PIPELINE ENCRYPTION TEST
=====

[Time           25000] INPUT FED: Packet 1
[Time           35000] INPUT FED: Packet 2
[Time           45000] INPUT FED: Packet 3
[Time           55000] INPUT STOPPED)
>> AES-128 | Packet 1 (Time:           135000 ns)
-----
Plaintext       : 3243f6a8885a308d313198a2e0370734
Key             : 2b7e151628aed2a6abf7158809cf4f3c
Ciphertext (DUT) : 3925841d02dc09fbdcl18597196a0b32
Expected Ciphertext: 3925841d02dc09fbdcl18597196a0b32
Result          : PASS

>> AES-128 | Packet 2 (Time:           145000 ns)
-----
Plaintext       : 00112233445566778899aabbccddeeff
Key             : 2b7e151628aed2a6abf7158809cf4f3c
Ciphertext (DUT) : 8df4e9aac5c7573a27d8d055d6e4d64b
Expected Ciphertext: 7649abac8119b246cee98e9b12e9197d
Result          : FAIL

>> AES-128 | Packet 3 (Time:           155000 ns)
-----
Plaintext       : 6bc1bee22e409f96e93d7e117393172a
Key             : 2b7e151628aed2a6abf7158809cf4f3c
Ciphertext (DUT) : 3ad77bb40d7a3660a89ecaf32466ef97
Expected Ciphertext: 3ad77bb40d7a3660a89ecaf32466ef97
Result          : PASS
```

Hình 6.4: Kết quả mô phỏng cho kiến trúc Pipeline với key 128-bit

```
>> AES-192 | Packet 1 (Time:           155000 ns)
-----
Plaintext       : 3243f6a8885a308d313198a2e0370734
Key             : 000102030405060708090a0b0c0d0e0f1011121314151617
Ciphertext (DUT) : bc3aaab5d97baa7b325d7b8f69cd7ca8
Expected Ciphertext: bc3aaab5d97baa7b325d7b8f69cd7ca8
Result          : PASS

>> AES-192 | Packet 2 (Time:           165000 ns)
-----
Plaintext       : 00112233445566778899aabbccddeeff
Key             : 000102030405060708090a0b0c0d0e0f1011121314151617
Ciphertext (DUT) : dda97ca4864cdf06eaf70a0ec0d7191
Expected Ciphertext: dda97ca4864cdf06eaf70a0ec0d7191
Result          : PASS

>> AES-192 | Packet 3 (Time:           175000 ns)
-----
Plaintext       : 6bc1bee22e409f96e93d7e117393172a
Key             : 000102030405060708090a0b0c0d0e0f1011121314151617
Ciphertext (DUT) : 1b58bc54cd0cb07alc91b8d25339da3b
Expected Ciphertext: 1b58bc54cd0cb07alc91b8d25339da3b
Result          : PASS
```

Hình 6.5: Kết quả mô phỏng cho kiến trúc Pipeline với key 192-bit



```
>> AES-256 | Packet 1 (Time: 175000 ns)
-----
Plaintext      : 3243f6a8885a308d313198a2e0370734
Key            : 000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
Ciphertext (DUT) : 9a198830ff9a4e39ec1501547d4a6b1b
Expected Ciphertext: 9a198830ff9a4e39ec1501547d4a6b1b
Result         : PASS

>> AES-256 | Packet 2 (Time: 185000 ns)
-----
Plaintext      : 00112233445566778899aabbccddeeff
Key            : 000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
Ciphertext (DUT) : 8ea2b7ca516745bfeafc49904b496089
Expected Ciphertext: 8ea2b7ca516745bfeafc49904b496089
Result         : PASS

>> AES-256 | Packet 3 (Time: 195000 ns)
-----
Plaintext      : 6bc1bee22e409f96e93d7e117393172a
Key            : 000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
Ciphertext (DUT) : e0a8f50ec76a04d5a96a175aa870ef63
Expected Ciphertext: e0a8f50ec76a04d5a96a175aa870ef63
Result         : PASS

=====
ENCRIPTION TEST COMPLETED
=====
```

Hình 6.6: Kết quả mô phỏng cho kiến trúc Pipeline với key 256-bit

Kết quả mô phỏng khi truyền vào 3 gói tin hiệu liên tiếp, thì kết quả được cho ra liên tiếp cho thấy hiệu suất mã hóa đã được tăng lên vượt trội. Bên cạnh đó kết quả cũng chính xác.

7 Tổng hợp và so sánh kết quả

7.1 Tổng quan về phần tổng hợp

Để thấy rõ hơn kết quả, việc tổng hợp (Synthesis) và hiện thực (Implementation) sẽ mang đến cái nhìn rõ ràng hơn phần cứng mà mã nguồn hiện thực đã tổng hợp được. Phần mềm tổng hợp bộ mã hóa AES này là Vivado với FPGA được chọn để chạy tổng hợp là **kria KV260**.

Thiết kế lõi AES hiện tại yêu cầu băng thông dữ liệu vào/ra rất lớn bao gồm 128-bit Input, 128-bit Key và 128-bit Output. Tổng số lượng chân tín hiệu yêu cầu lên tới ≈ 400 chân. Con số này vượt quá số lượng chân vật lý khả dụng của dòng chip FPGA hiện tại. Do đó, việc cố gắng tổng hợp toàn mạch ở chế độ thông thường sẽ dẫn đến lỗi "I/O Placement Failure".

Để giải quyết vấn đề này và đảm bảo tính chính xác khi đánh giá tài nguyên, đồ án áp dụng quy trình tổng hợp **Out-of-Context (OOC)**. Phương pháp này mang lại các lợi ích kỹ thuật quan trọng:

- **Khắc phục giới hạn vật lý:** Cho phép tổng hợp logic lõi mà không cần gán chân vật lý thực tế, giúp đánh giá tài nguyên thuật toán độc lập với phần cứng ngoại vi.

- **Đảm bảo tính độc lập:** Module lõi AES được cô lập và tổng hợp độc lập với các lớp giao tiếp và chân tín hiệu vật lý (I/O Pads). Phương pháp này giúp loại bỏ các sai số do bộ đệm gây ra, đồng thời cho phép phân tích thời gian của riêng kiến trúc được tổng hợp một cách chính xác nhất."

Quá trình tổng hợp và hiện thực các kiến trúc trên Vivado với clock nằm trong file ràng buộc (constraint) để tạo xung clock cấp cho quy trình tổng hợp **OOC** như sau:

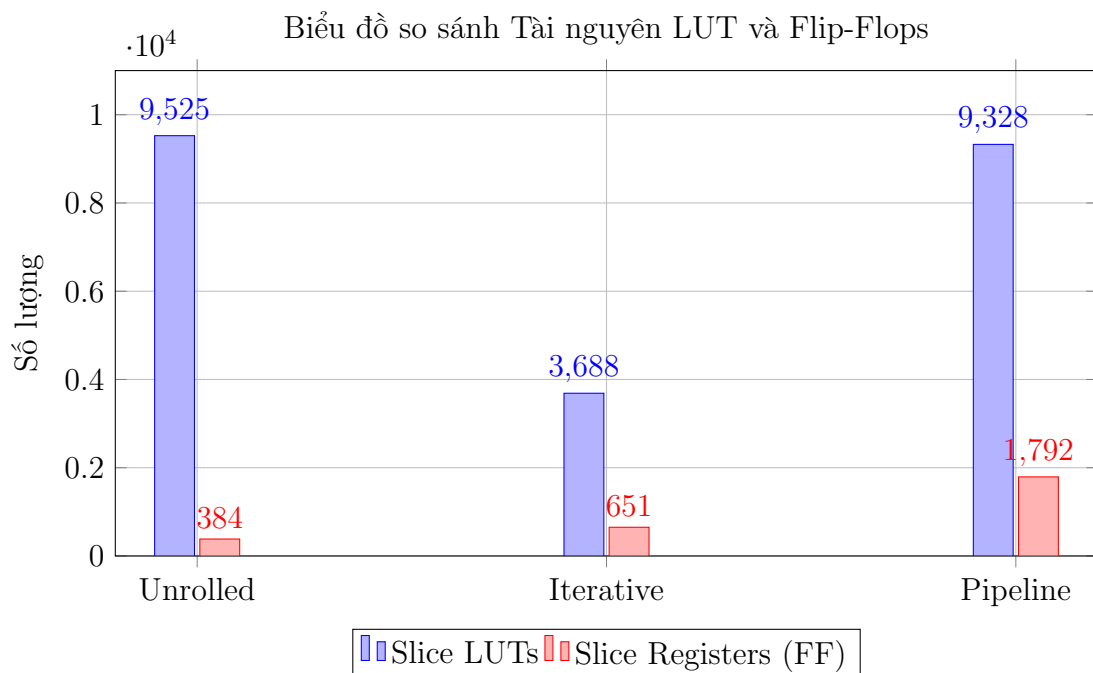
```
create_clock -period 10.000 -name sys_clk -waveform {0.000 5.000}
[get_ports clk]
```

File ràng buộc này tạo ra một xung nhịp với tần số 100MHz với ý nghĩa là yêu cầu Vivado sắp xếp linh kiện hợp lý sao cho đường đi của tín hiệu chỉ nằm trong 1 chu kỳ của xung này, qua đó giúp cung cấp thông tin về thời gian xử lý tín hiệu thông qua kết quả Implementation.

Mục tiêu của phần này là đánh giá và so sánh 3 kiến trúc về tài nguyên sử dụng, hiệu suất thời gian và công suất điện tiêu thụ.

7.2 Tài nguyên phần cứng

Số liệu tiêu thụ tài nguyên phần cứng được trích xuất từ báo cáo sau giai đoạn Implementation:



Hình 7.1: Biểu đồ so sánh tài nguyên tiêu thụ giữa 3 kiến trúc

Phân tích kết quả:

- **Slice LUTs (Logic):** Kiến trúc **Iterative** chiếm ít tài nguyên logic nhất (3688 LUTs) do tái sử dụng một khối phần cứng duy nhất cho 10 vòng lặp. Ngược lại, **Unrolled** (9525 LUTs) và **Pipeline** (9328 LUTs) tiêu tốn tài nguyên gấp khoảng 2.5 lần do nhân bản logic của 10 vòng mã hóa.
- **Slice Registers (Flip-Flops):**
 - **Unrolled** sử dụng ít FF nhất (384) vì bản chất là mạch tổ hợp thuần túy giữa đầu vào và đầu ra.
 - **Pipeline** sử dụng lượng FF vượt trội (1792), cao gấp gần 3 lần Iterative. Điều này phản ánh chính xác chi phí cho các thanh ghi được chèn vào giữa để lưu kết quả của từng bước trong pipeline.

7.3 Hiệu suất thời gian

Dựa trên tham số WNS (Worst Negative Slack) từ báo cáo Timing với ràng buộc chu kỳ $T_{target} = 10\text{ns}$ (100MHz), ta tính toán được tần số tối đa (F_{max}) và thông lượng (Throughput).

Công thức tính toán:

$$F_{max} = \frac{1}{T_{target} - \text{WNS}} \quad ; \quad \text{Throughput} = \frac{128 \times F_{max}}{\text{Cycles/Block}} \quad (1)$$

Bảng 4: Bảng tổng hợp hiệu năng thời gian

Kiến trúc	WNS (ns)	T_{min} (ns)	F_{max} (MHz)	Cycles/Block	Throughput (Mbps)
Unrolled	-9.334	19.334	51.72	1	6,620
Iterative	-1.461	11.461	87.25	11	1,015
Pipeline	-2.976	12.976	77.06	1	9,864

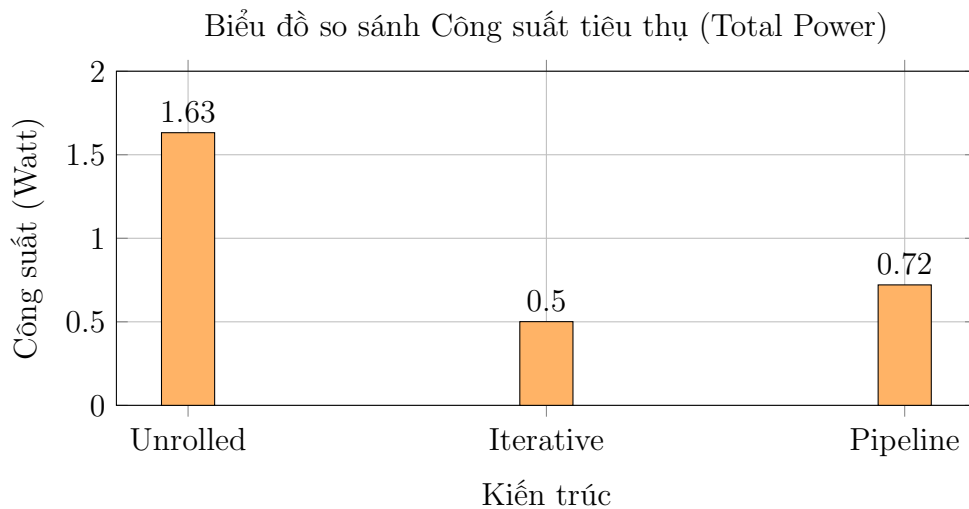
Nhận xét và Đánh giá:

- **Về tần số (F_{max}):**
 - **Unrolled** có F_{max} thấp nhất (≈ 51.7 MHz) do đường tới hạn (Critical Path) kéo dài xuyên suốt 10 vòng mã hóa mà không có thanh ghi cắt ngang.
 - **Iterative** đạt F_{max} cao nhất (≈ 87.2 MHz) nhờ mạch gọn nhẹ, ít bị trễ do dây dẫn (Routing Delay).
 - **Pipeline** có F_{max} thấp hơn Iterative một chút (≈ 77 MHz). Nguyên nhân là do diện tích mạch lớn, có thêm nhiều thanh ghi khiến đường đi của tín hiệu dài.
- **Về Thông lượng (Throughput):**

- Mặc dù F_{max} không cao nhất, kiến trúc **Pipeline** đạt thông lượng vô địch (≈ 9.86 Gbps), gấp gần 10 lần so với Iterative. Đây là minh chứng cho hiệu quả của kỹ thuật xử lý song song mức thời gian.
- **Iterative** có thông lượng thấp nhất (≈ 1 Gbps) do phải mất 11 chu kỳ xung nhịp để xử lý xong một khối dữ liệu.

7.4 Công suất tiêu thụ (Power Consumption)

Số liệu tổng công suất tiêu thụ của từng kiến trúc như sau:



Hình 7.2: Công suất tiêu thụ của các kiến trúc

Phân tích:

- **Iterative (0.501 W):** Tiêu thụ năng lượng thấp nhất, tương ứng với việc sử dụng ít tài nguyên nhất.
- **Unrolled (1.632 W):** Có công suất tiêu thụ cao bất thường (gấp 3 lần Iterative). Nguyên nhân chính là do hiện tượng **Glitching** (nhiều gai) trong khối logic tổ hợp khổng lồ. Do không có thanh ghi chốt giữ trạng thái trung gian, các tín hiệu chuyển mạch liên tục và lan truyền tự do gây tiêu hao năng lượng động rất lớn.
- **Pipeline (0.721 W):** Mức tiêu thụ năng lượng ở mức trung bình. Mặc dù có số lượng Flip-Flop lớn, nhưng nhờ các thanh ghi này chặn đứng Glitching giữa các tầng, năng lượng động được kiểm soát tốt hơn so với Unrolled.

Bảng so sánh tổng hợp: Bảng 5 tóm tắt các đặc tính kỹ thuật của ba kiến trúc dựa trên kết quả thực nghiệm trên dòng chip FPGA mục tiêu.

Kết quả thực nghiệm cho thấy sự đánh đổi rõ rệt giữa tài nguyên và hiệu năng, từ đó định hình phạm vi ứng dụng cho từng kiến trúc:



Bảng 5: Bảng so sánh đặc tính kỹ thuật giữa các kiến trúc AES-128

Tiêu chí	Iterative	Pipeline	Unrolled
Diện tích (LUTs / FFs)	Thấp nhất ($\approx 3.6k / 651$)	Cao ($\approx 9.3k / 1792$)	Cao ($\approx 9.5k / 384$)
Tần số (F_{max}) (Timing Closure)	Cao (≈ 87 MHz)	Trung bình (≈ 77 MHz)	Thấp (≈ 51 MHz)
Thông lượng (Throughput) (Tốc độ xử lý)	Thấp (≈ 1.0 Gbps)	Rất cao (≈ 9.8 Gbps)	Trung bình (≈ 6.6 Gbps)
Độ trễ (Latency) (Cycles)	Trung bình (11 Cycles)	Cao (10 Cycles)	Thấp nhất (1 Cycle)
Công suất (Power) (Total On-chip Power)	Thấp nhất (0.501 W)	Trung bình (0.721 W)	Cao nhất (1.632 W)
Hiệu suất (Efficiency) (Throughput / Area)	Tốt (Cân bằng)	Tốt nhất (Tối ưu bằng thông)	Kém (Tốn diện tích)

- **Iterative:** Tối ưu hóa diện tích và công suất tiêu thụ (chỉ 0.5W), phù hợp nhất với các thiết bị tài nguyên hạn chế như **IoT**.
- **Pipeline:** Đạt thông lượng vượt trội (≈ 9.8 Gbps), là lựa chọn lý tưởng cho các hệ thống yêu cầu băng thông lớn như **mạng viễn thông, trung tâm dữ liệu**.
- **Unrolled:** Có độ trễ xử lý thấp nhất (1 chu kỳ), thích hợp cho các ứng dụng điều khiển phản hồi tức thì đòi hỏi tính **thời gian thực (Real-time)** khắt khe.

8 Kết luận và Hướng phát triển

Đồ án đã hoàn thành việc thiết kế, hiện thực và đánh giá ba kiến trúc AES, chỉ ra sự đánh đổi rõ rệt giữa diện tích và hiệu suất.

Dựa trên nền tảng này, hướng phát triển tiếp theo của đề tài là tích hợp lõi IP AES cùng với **bộ tăng tốc CNN (Convolutional Neural Network)** trên cùng một hệ thống **SoC**. Mục tiêu nghiên cứu là phân tích hiệu năng thực tế và mức độ tiêu thụ năng lượng khi hai tác vụ bảo mật dữ liệu và xử lý trí tuệ nhân tạo hoạt động song song trên cùng một chip FPGA.

Link toàn bộ Source code: [Click here](#)



Tài liệu tham khảo

- [1] N. I. of S. A. Technology, “Advanced Encryption Standard (AES),” May 2023. doi: 10.6028/nist.fips.197-upd1.